

Assignment 6 :

GitHub Link :

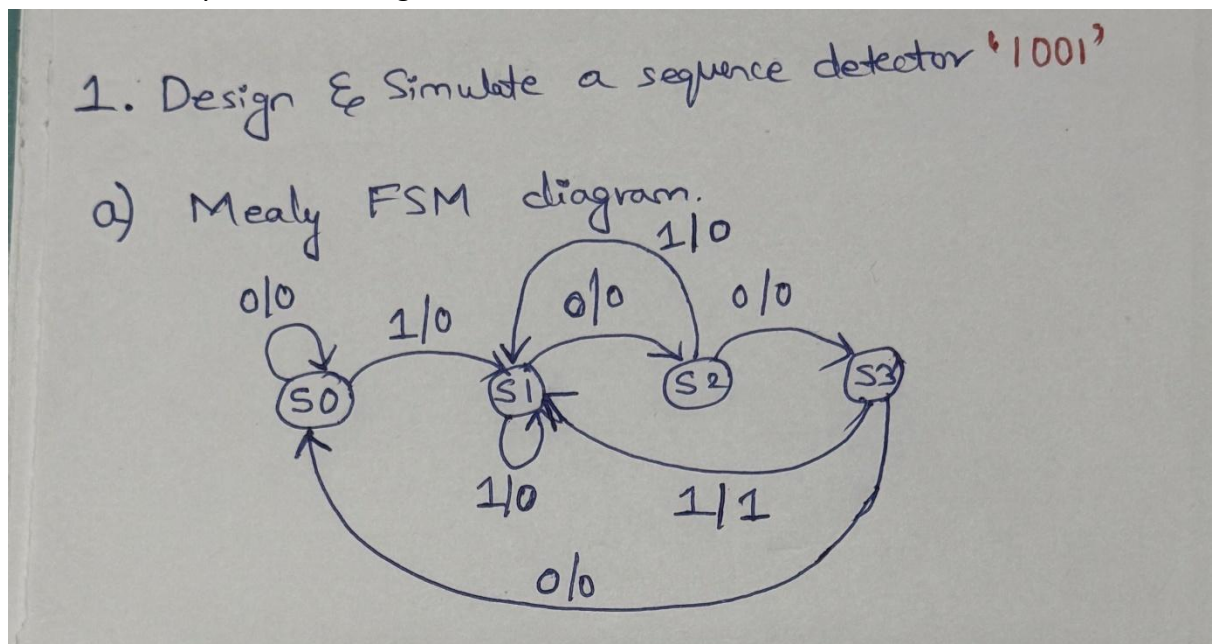
https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/tree/main/DAY3/Assignment%206

README.md Link :

https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/blob/main/DAY3/Assignment%206/README.md

Question : Design and simulate a sequence detector for detecting '1001'.

1. Draw the Mealy FSM state diagram.



2. Implement the Mealy FSM in Verilog HDL.

Verilog Code :

```
`timescale 1ns/1ps

module mealy_fsm_detect_1001 (
    input wire clk,
    input wire reset,
    input wire data_in,
    output reg detected
);

    localparam [1:0] S0 = 2'b00,
                                     S1 = 2'b01,
                                     S2 = 2'b10,
                                     S3 = 2'b11;

    reg [1:0] PS, NS;

    always @(posedge clk or posedge reset) begin
        if (reset)
            PS <= S0;
        else
            PS <= NS;
    end

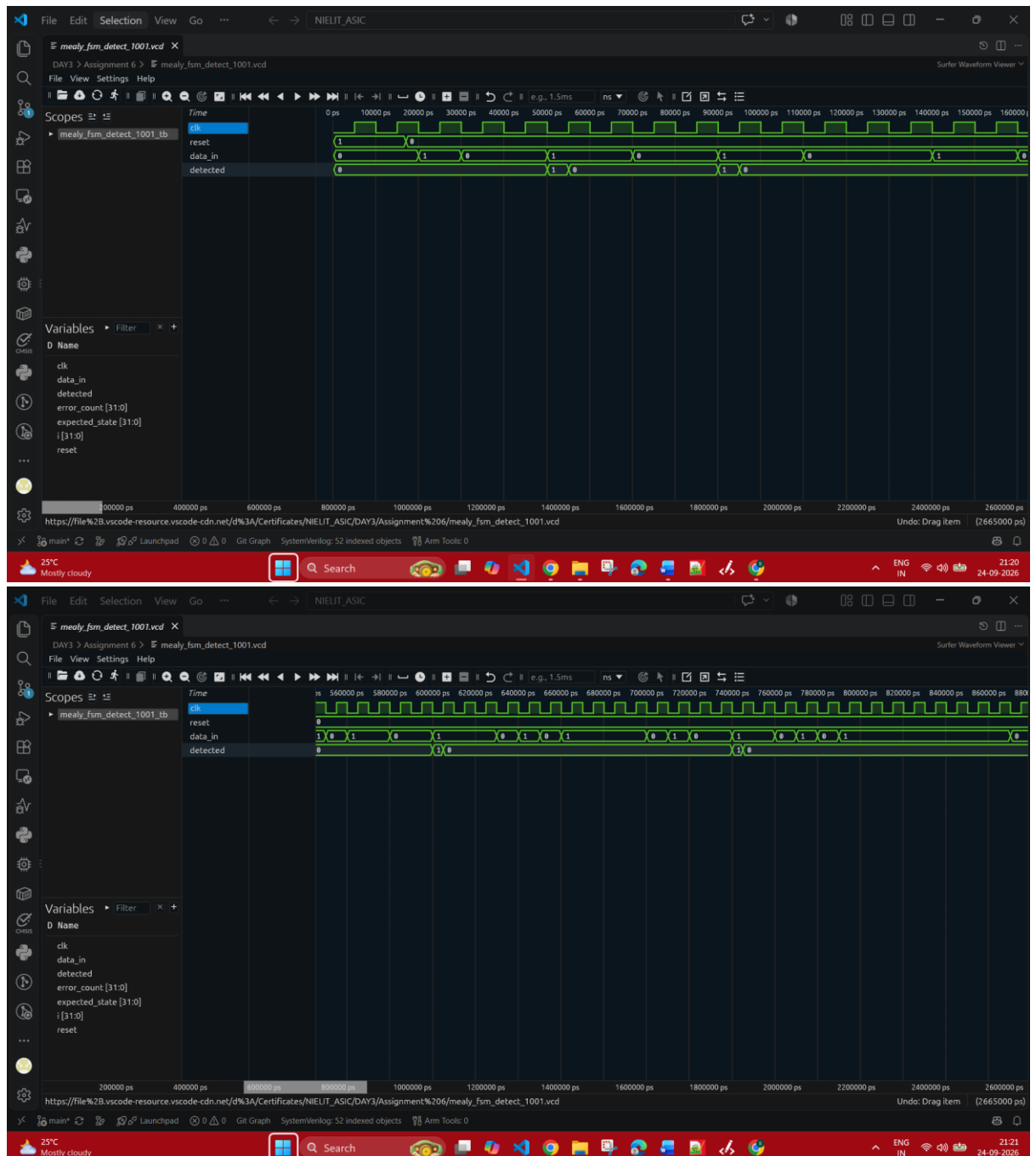
    always @(PS or data_in) begin
        NS = S0;

        case (PS)
            S0: begin
                NS = data_in ? S1 : S0;
            end

            S1: begin
                NS = data_in ? S1 : S2;
            end

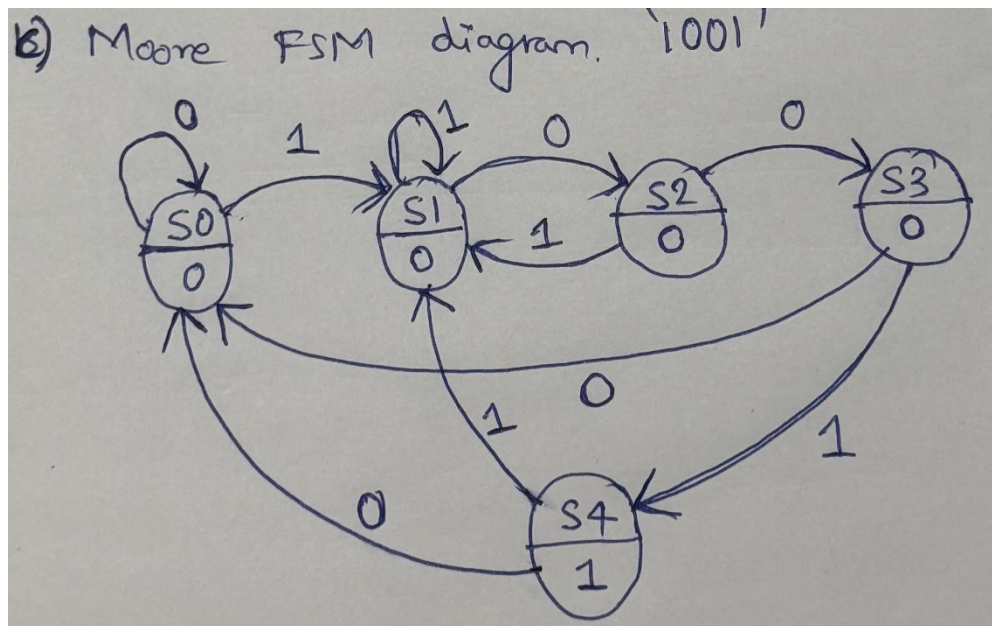
            S2: begin
                NS = data_in ? S3 : S0;
            end
        end
    end
endmodule
```


Simulation Log :



3. Draw the Moore FSM state diagram.

Non-Overlapping Case :



4. Implement the Mealy FSM in Verilog HDL.

Verilog Code :

```
`timescale 1ns/1ps
```

```
module moore_fsm_detect_1001_nonoverlap (
```

```
    input wire clk,
```

```
    input wire reset,
```

```
    input wire data_in,
```

```
    output reg detected
```

```
);
```

```
    localparam [2:0] S0 = 3'b000,
```

```
                S1 = 3'b001,
```

```
                S2 = 3'b010,
```

```
                S3 = 3'b011,
```

```
                S4 = 3'b100;
```

```
    reg [2:0] PS, NS;
```

```
    always @(posedge clk or posedge reset) begin
```

```
        if (reset)
```

```
            PS <= S0;
```

```

        else
            PS <= NS;
        end

always @(PS, data_in) begin
    NS = S0;

    case (PS)
        S0: begin
            NS = data_in ? S1 : S0;
        end

        S1: begin
            NS = data_in ? S1 : S2;
        end

        S2: begin
            NS = data_in ? S1 : S3;
        end

        S3: begin
            NS = data_in ? S4 : S0;
        end

        S4: begin
            NS = data_in ? S1 : S0;
        end

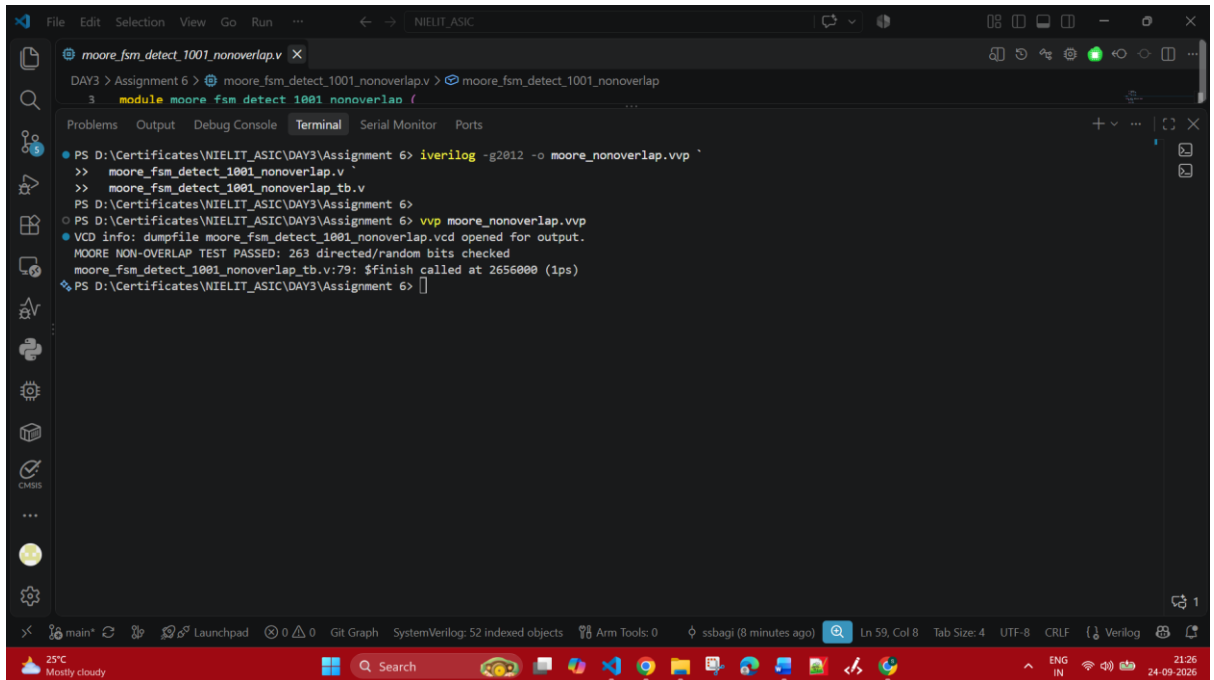
        default: NS = S0;
    endcase
end

always @(PS) begin
    case (PS)
        S0, S1, S2, S3: detected = 1'b0;
        S4: detected = 1'b1;
        default: detected = 1'b0;
    endcase
end

endmodule

```

Compilation Log :

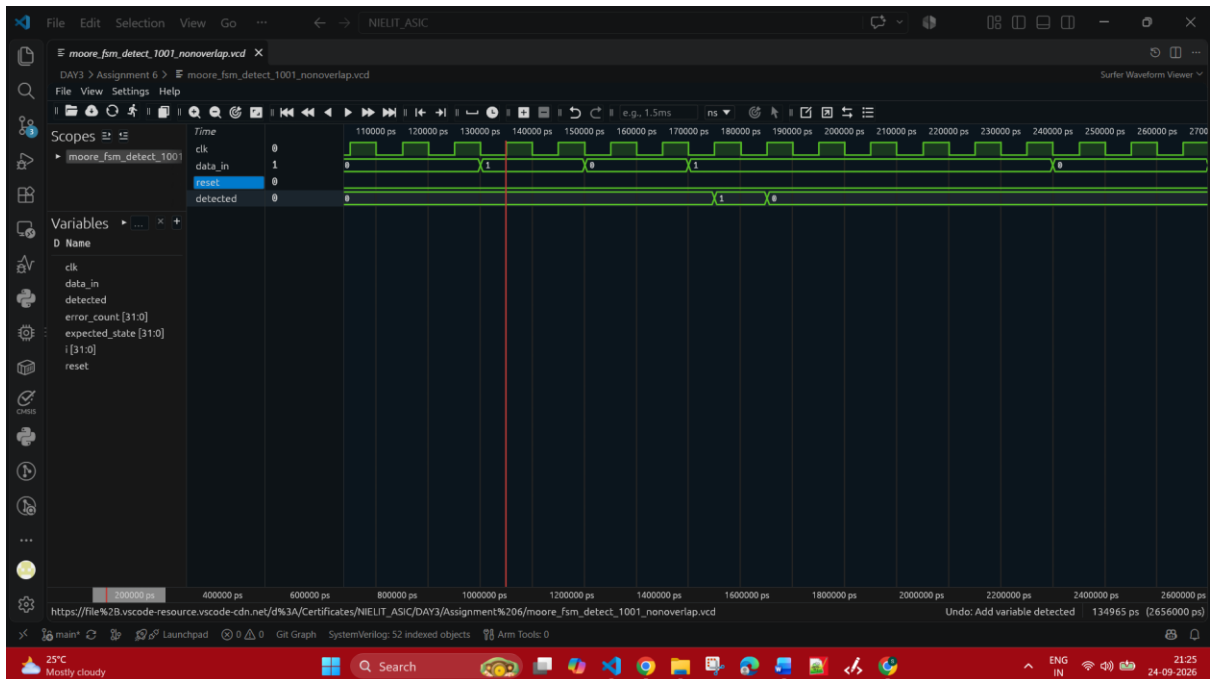


The screenshot shows the Verilog compilation log in VS Code. The log is displayed in the 'Terminal' tab, showing the output of the 'iverilog' command. The log includes the following information:

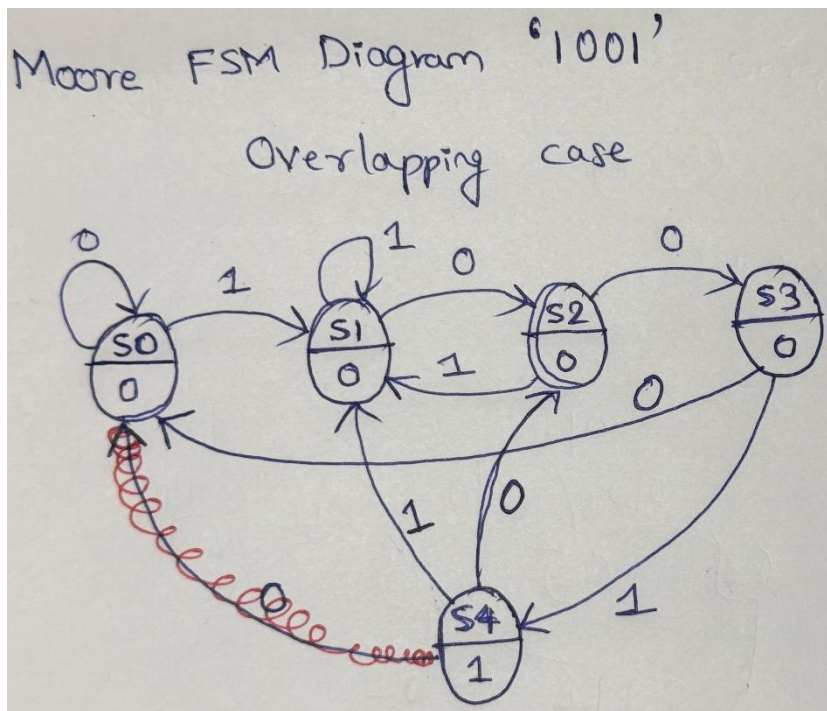
- PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6> iverilog -g2012 -o moore_nonoverlap.vvp
- >> moore_fsm_detect_1001_nonoverlap.v
- >> moore_fsm_detect_1001_nonoverlap_tb.v
- PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6>
- PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6> vvp moore_nonoverlap.vvp
- VCD info: dumpfile moore_fsm_detect_1001_nonoverlap.vcd opened for output.
- MOORE NON-OVERLAP TEST PASSED: 263 directed/random bits checked
- moore_fsm_detect_1001_nonoverlap_tb.v:79: \$finish called at 2656000 (1ps)
- PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6>

The log also shows the file explorer on the left, with the 'moore_fsm_detect_1001_nonoverlap.v' file selected. The status bar at the bottom indicates the file is 'Ln 59, Col 8' and the tab size is '4'.

Simulation Log :



Overlapping Case :



Verilog code :

```
`timescale 1ns/1ps

module moore_fsm_detect_1001_overlap (

    input wire clk,

    input wire reset,

    input wire data_in,

    output reg detected

);

    localparam [2:0] S0 = 3'b000,

                                S1 = 3'b001,

                                S2 = 3'b010,

                                S3 = 3'b011,

                                S4 = 3'b100;

    reg [2:0] PS, NS;

    always @(posedge clk or posedge reset) begin
        if (reset)
            PS <= S0;
        else
            PS <= NS;
    end

    always @(PS, data_in) begin

        NS = S0;

        case (PS)
```

```

        S0: begin
            NS = data_in ? S1 : S0;
        end

        S1: begin
            NS = data_in ? S1 : S2;
        end

        S2: begin
            NS = data_in ? S1 : S3;
        end

        S3: begin
            NS = data_in ? S4 : S0;
        end

        S4: begin
            NS = data_in ? S1 : S2;
        end

        default: NS = S0;
    endcase
end

always @(PS) begin
    case (PS)
        S0, S1, S2, S3: detected = 1'b0;
        S4: detected = 1'b1;
    endcase
end

```

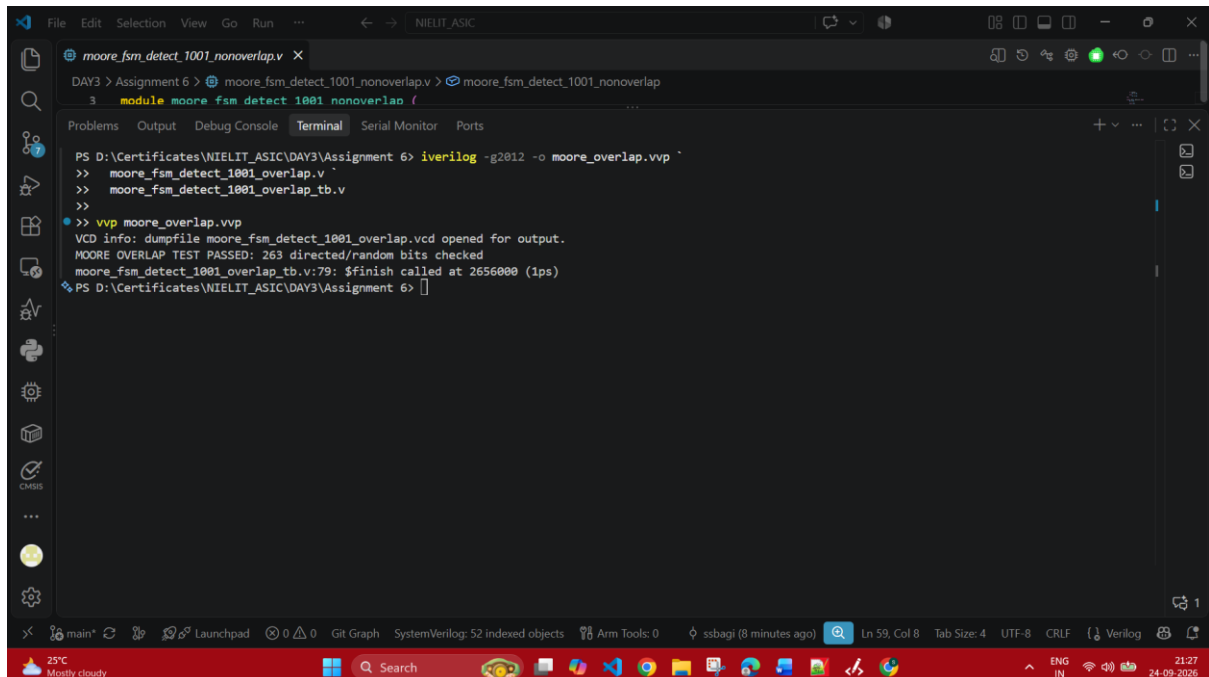
```
default: detected = 1'b0;
```

```
endcase
```

```
end
```

```
endmodule
```

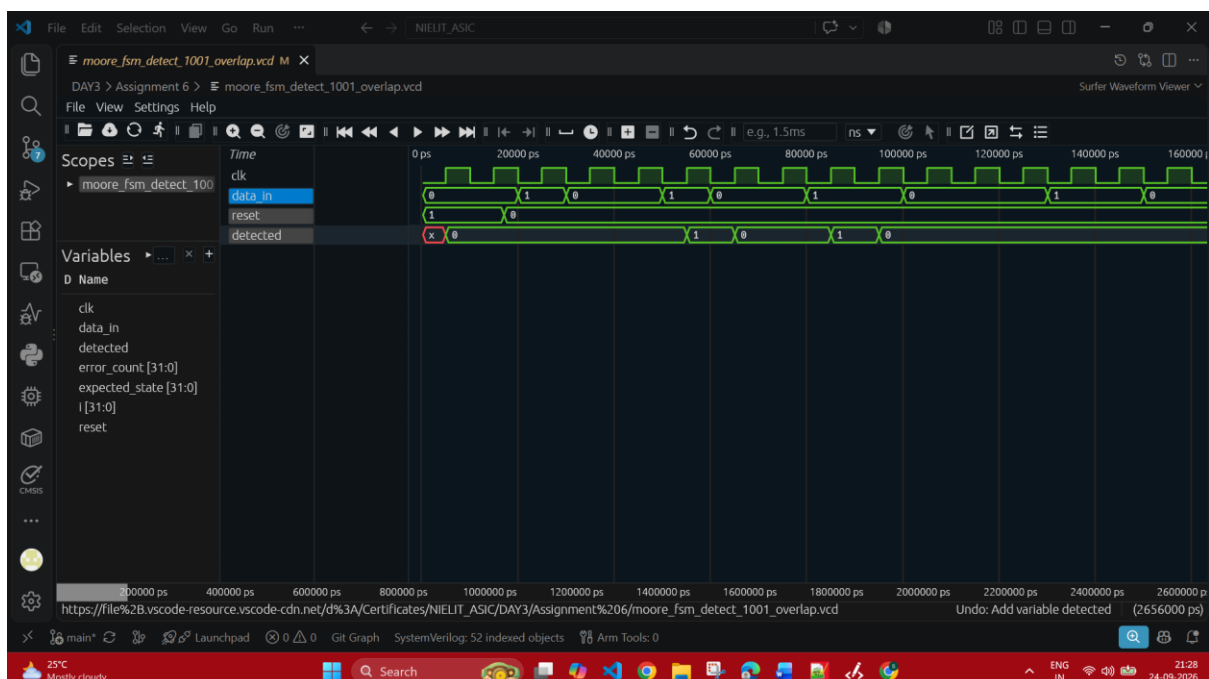
Compilation Log :



The screenshot shows a terminal window with the following commands and output:

```
PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6> iverilog -g2012 -o moore_overlap.vvp `
>> moore_fsm_detect_1001_overlap.v `
>> moore_fsm_detect_1001_overlap_tb.v
>>
>> vvp moore_overlap.vvp
VCD info: dumpfile moore_fsm_detect_1001_overlap.vcd opened for output.
MOORE OVERLAP TEST PASSED: 263 directed/random bits checked
moore_fsm_detect_1001_overlap_tb.v:79: $finish called at 2656000 (1ps)
PS D:\Certificates\NIELIT_ASIC\DAY3\Assignment 6>
```

Simulation Log :



5. Compare Mealy and Moore FSM implementations.

Mealy Machine :

- The output depends on Present state and Past Input. [always @(PS or data_in)]
- Unregister output and no extra state

Moore Machine :

- The output depends only on Present state. [always @(PS)]
- Many more additional states and registered output.